Technical Design Document

Dancing Cactus Studio

Game: Remains

Zijing Liu (Kevin)

Technical Design Flowchart Link :
https://www.figma.com/board/FDKqwLwmx6aFHmNpCc7856/Remains-Technical-Design?node-id=0-1&t=thDETtDewRmJt8eC-1

Contents

1 Tech Design 4

1.1 Spline Movement: 4

1.1.1 General Idea: 4

1.1.2 Game Programming Pattern: 4

1.1.3 Technical Design Flowchart 7

1.2 Camera Control 7

1.2.1 General Idea: 7

1.2.2 Game Programming Pattern 8

1.2.3 Technical Design Flowchart 8

1.3 Player Power System 8

1.3.1 General Ideas: 8

1.3.2 Game Programming Pattern: 9

1.3.3 Technical Design Flowchart 14

1.4 Level Progression & Checkpoint System 14

1.4.1 General Ideas 14

1.4.2 Game Programming Patterns 14

1.4.3 Technical Design Flowchart 18

1.5 Key & Level Unlock System 18

1.5.1 General Ideas 18

1.5.2 Game Programming Pattern 18

1.5.3 Game Design Flowchart 20

1.6 Weapon System 20

1.6.1 General Ideas 20

1.6.2 Game Programming Pattern 20

1.6.3 Technical Design Flowchart 25

1.7 NPC and Enemy State Machine System 25

1.7.1 General Ideas 25

1.7.2 Game Programming Pattern 25

1.7.3 Technical Design Flowchart 33

1.8 Reaction Panel 35

1.8.1 General Ideas 35

1.8.2 Service Oriented Architecture 35

1.8.3 Game Programming Pattern 35
1.8.4 Technical Design Flowchart 38
1.9 Player Health System 38
1.9.1 General Ideas 38
1.9.2 Game Programming Pattern 38
1.9.3 Technical Design Flowchart 43
1.10 Player Dirty and Clean System 43
1.10.1 General Ideas 43
1.10.2 Game Programming Pattern 43
1.10.3 Technical Design Flowchart 46
1.11 Slippery and Water Zone 47
1.11.1 General Ideas 47
1.11.2 Game Programming Pattern 47
1.11.3 Technical Design Flowchart 50
1.12 Setting Controls 50
1.12.1 General Ideas 50
1.12.2 Game Programming Pattern 51
1.12.3 Technical Design Flowchart 57
1.13 Battery Pick Up and Trigger Elevator 57
1.13.1 General Ideas 57
1.13.2 Game Programming Patterns 57
1.13.3 Technical Design Flowchart 59
1.14 Tutorial Instruction System 59
1.14.1 General Ideas 59
1.14.2 Game Programming Pattern 59
1.14.3 Technical Design Flowchart 61

1 Tech Design

1.1 Spline Movement:

1.1.1 General Idea:

The mechanic requirement is to let the game objects, especially the player to move along a customized track made by developer, we used the Unity Spline Tool to draw the track. So make the game different from other side scroller platformer games whose tracks are always straight lines. Our game track will be customized like a curvy or circular line.

1.1.2 Game Programming Pattern:

Component Pattern:

The movement system follows Unity's Component-based architecture. Each responsibility is separated into individual components that are composed on GameObject instead of relying on inheritance.

SplineTracker: dealing with data and math.

SplineObjectMover: make sure the game object move along the spline track.

CollisionSweeper: forecasting the potential collision

SplineMovableObject: auto-move along the spline and behavior of getting pushed

SplineRunnerRB: manually controlling player movement by input. Not spline math

Service / Façade Pattern:

SplineTracker acts as Façade over Unity's spline API, exposing a small, cohesive set of methods for movement and projection. In code, the gameplay scripts don't call SplineUtility or spline matrices

directly, they call SplineTracker. CollisionSweeper provides a collision sweeping service that abstracts away the details of different collider types.

SplineTracker.MoveAlongSpline() convers speed to normalized spline *t* using an approximate length.

```
public float MoveAlongSpline(float currentT, float speedMetersPerSec, float deltaTime, float direction)
{
    if (!ValidateAndCacheSpline()) return currentT;
    if (!math.isfinite(_approxLength) || _approxLength < 0.001f) RecomputeLength();

    float dtAsT = (speedMetersPerSec * deltaTime) / math.max(0.001f, _approxLength);
    float newT = currentT + direction * dtAsT;

    newT = loop ? Mathf.Repeat(newT, 1f) : Mathf.Clamp01(newT);
    return math.isfinite(newT) ? newT : currentT;
}
```

Projection façade (world – nearest spline point)

```
public SplineProjectionResult ProjectWorldPointToSpline(Vector3 worldPoint)
{
    if (!ValidateAndCacheSpline())
        return new SplineProjectionResult { t = 0f, worldPosition = worldPoint, isValid = false };

    RefreshMatrices();
    float3 localPoint = math.transform(_worldToLocalMatrix, (float3)worldPoint);
    SplineUtility.GetNearestPoint(_spline, localPoint, out float3 nearestLocal, out float t);

    float3 nearestWorld = math.transform(_worldMatrix, nearestLocal);

    return new SplineProjectionResult
    {
        t = t,
        worldPosition = (Vector3)nearestWorld,
        isValid = true
    };
}
```

By doing this, callers like SplineRunnerRB, SplineObjectMover work with simple concepts of speed, direction and nearest point. Unity Splines complexity (local, world transforms, SplineUtility) is contained in one place. And this makes camera and other movable objects can reuse the same spline API surface consistently.

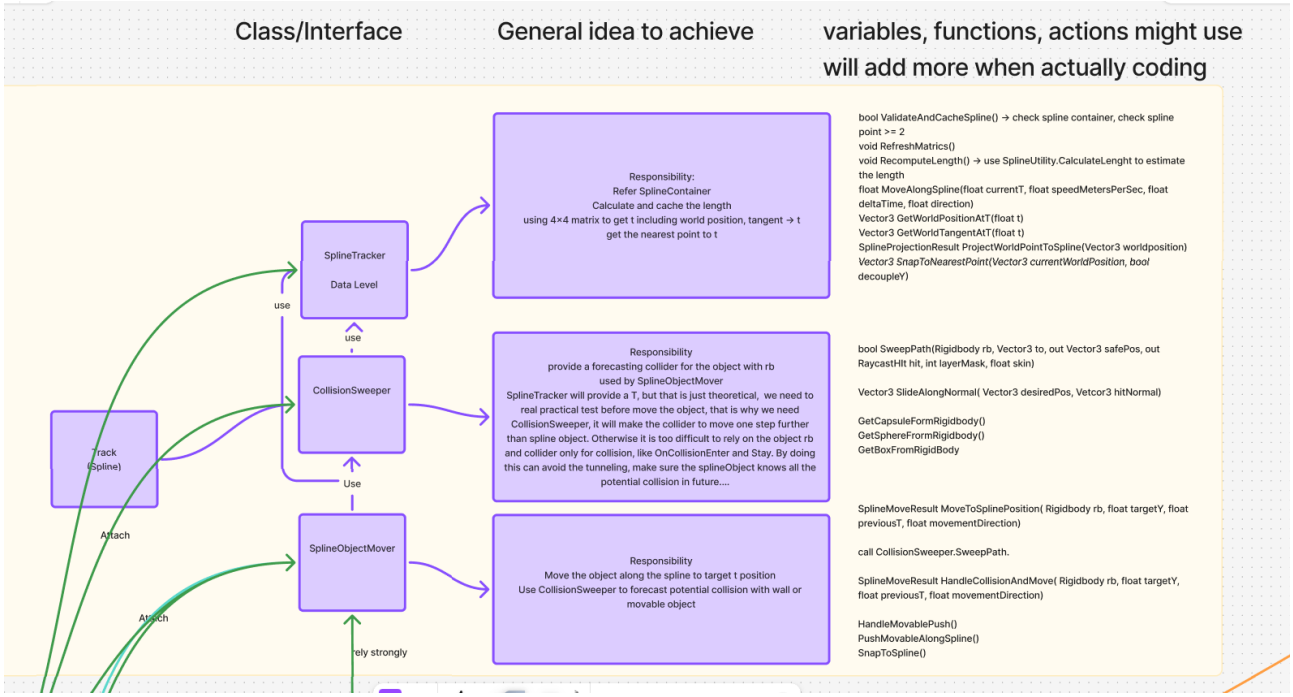
Observer Pattern:

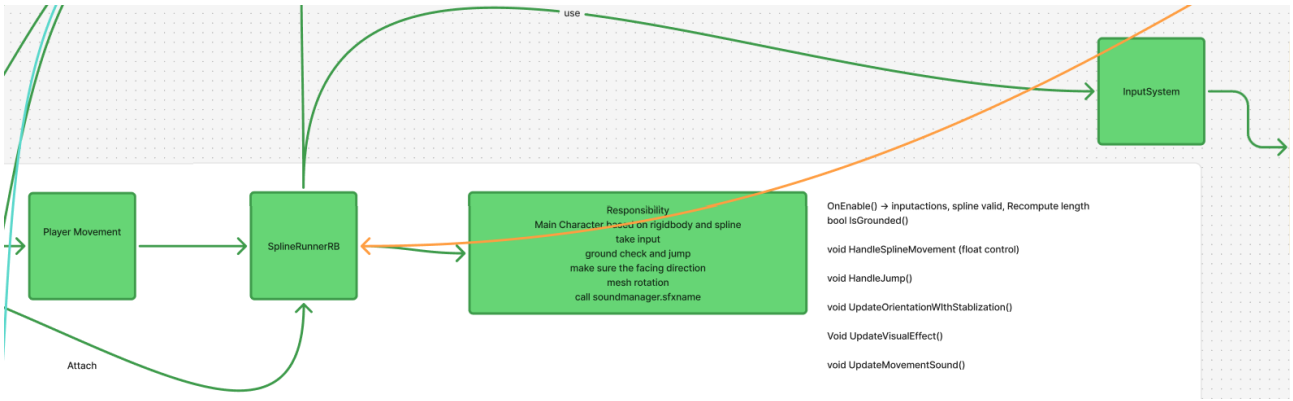
Using events in SplineMovableObject, exposing several UnityEvent callbacks, like SplineMovableObject.OnReachSplineStart, SplineMovableObject.OnReachSplineEnd, SplineMovableObject.OnPushed for other scripts to subscribe.

```
[Header("Events")]
public UnityEngine.Events.UnityEvent OnReachSplineStart;
public UnityEngine.Events.UnityEvent OnReachSplineEnd;
public UnityEngine.Events.UnityEvent<float> OnPushed;
```

```
public bool TryPush(float pushDistance, Rigidbody pusher = null)
{
    // compute effective push, compute newT, move via SplineObjectMover...
    if (moveResult.success)
    {
        _t = moveResult.finalT;
        OnPushed?.Invoke(Mathf.Abs(effectivePush));
        return true;
    }
    return false;
}
```

1.1.3 Technical Design Flowchart





1.2 Camera Control

1.2.1 General Idea:

The game is a side scroller view so the camera should only focus on one side of the player. We tried only cinemachine camera firstly, the 3rd person follow camera but the built in virtual camera will always move to the back of the player when player turns direction in the circular path. Also, when the player turns the face around, the camera should not rotate. Furthermore, because our player moves along the spline which need to recompute the direction every frame, so if the “LookAt” is assigned to player, the camera will shaking. So we need a camera which is also moving along the customized spline track and keep a certain distance or offset with player. The solving

idea is that I will make the camera follows another game object which listens to the move input action as well, and the camera itself will get the spline track from player. In summary, I made a new rail camera but fully compatible with our game.

1.2.2 Game Programming Pattern

Component-based Pattern:

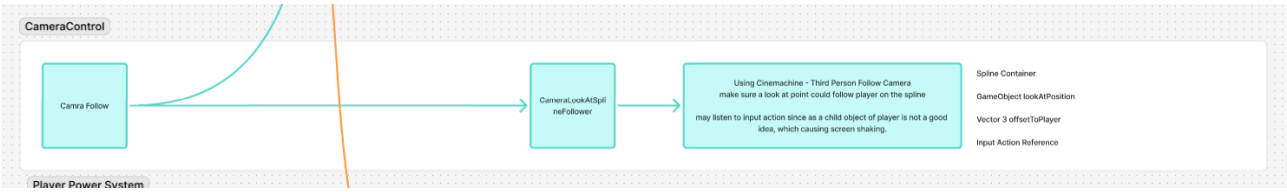
This script acts as a camera controller component that positions the camera along the same spline as the player, maintaining a configurable offset and height.

Service / Façade Pattern:

This script does not rely on the Unity Splines API, but through SplineTracker to evaluate, projection, move along spline.

```
var proj = splineTracker.ProjectWorldPointToSpline(transform.position);
_t = splineTracker.MoveAlongSpline(_t, proxySpeed, Time.deltaTime, inputDir);
transform.position = splineTracker.EvaluatePositionWorld(_t);
transform.rotation = Quaternion.LookRotation(splineTracker.EvaluateTangentWorld(_t));
```

1.2.3 Technical Design Flowchart



1.3 Player Power System

1.3.1 General Ideas:

Player power will increase when the player enter or stay in the sun zone, the player power value should be accessed and changed by other systems, events will be used like OnPowerChanged. System will include the UI.

1.3.2 Game Programming Pattern:

Observer / Event Pattern:

PlayerPower exposes a C# event OnPowerChanged(current, max). Multiple independent systems subscribe to it (UI, VFX/emission, reaction panel logic), so no subscriber needs to know about zones or other subscribers—they only react to the power state. This matches the design doc's claim that OnPowerChanged is listened by PowerReactionController, PowerUI, and HeadFaceEmissionByPowe

```
public class PlayerPower : MonoBehaviour
{
    public event Action<int, int> OnPowerChanged; // (current, max)

    void Awake()
    {
        currentPower = Mathf.Clamp(initialPower, 0, maxPower);
        OnPowerChanged?.Invoke(currentPower, maxPower);
    }

    public void ModifyPower(int amount)
    {
        if (amount == 0) return;

        int before = currentPower;
        currentPower = Mathf.Clamp(currentPower + amount, 0, maxPower);

        if (currentPower != before)
        {
            OnPowerChanged?.Invoke(currentPower, maxPower);
        }
    }
}
```

For UI subscriber (PowerUI)

```
public class PowerUI : MonoBehaviour
{
    void Start()
    {
        var playerPower = FindFirstObjectByType<PlayerPower>();
        powerSlider.interactable = false;
        playerPower.OnPowerChanged += UpdateUI;

        powerSlider.maxValue = playerPower.Max;
        powerSlider.value = playerPower.Current;
    }

    void UpdateUI(int current, int max)
    {
        powerSlider.maxValue = max;
        powerSlider.value = current;
    }
}
```

For VFX subscriber (HeadFaceEmmisionByPower)

```
void OnEnable()
{
    if (playerPower != null)
        playerPower.OnPowerChanged += OnPowerChanged;

    ApplyByPower(playerPower ? playerPower.Current : int.MaxValue);
}

void OnDisable()
{
    if (playerPower != null)
        playerPower.OnPowerChanged -= OnPowerChanged;
}

private void OnPowerChanged(int current, int max)
{
    ApplyByPower(current);
}
```

For other system like Reaction subscriber (PowerReactionController)

```
void OnEnable()
{
    playerPower.OnPowerChanged += OnPowerChanged;
    EvaluateImmediate(playerPower.Current, playerPower.Max);
}

void OnDisable()
{
    playerPower.OnPowerChanged -= OnPowerChanged;
}

private void OnPowerChanged(int current, int max)
{
    EvaluateImmediate(current, max);
}
```

Model View Controller (MVC-like) Pattern:

PlayerPower acts like Model+Controller (stores state + applies rules and updates), while PowerUI and HeadFaceEmmissionByPower act like Views that render the state only through OnPower-Changed.

The Model + Controller responsibilities in PlayerPower are owing currentPower, maxPower; providing a single modification entry point ModifyPower(int amount); trigging OnPowerChanged when state changes and applying periodic zone delta (tick-based update). By doing this, Views don’t compute power rules; they render. Zones don’t update UI; they only influence the power model through a clean API. And PlayerPower becomes the “single source of truth” for power.

```
void Update()
{
    timer += Time.deltaTime;
    if (timer >= updateInterval)
    {
        timer = 0f;
        if (_currentChosenZone != null)
            ModifyPower(_currentChosenZone.GetPowerDelta());
    }
}
```

Mediator

ZoneTrigger will not care about the power value and UI, it only deal with the collision. So decouple the power value from collision. By doing this, ZoneTrigger could be use for different power collision situations like shadow zone. Therefore, ZoneTrigger handling trigger enter/exit/stay. IPowerZoneEffect representing “what this zone does to power”. SunZone / ShadowZone implementing the effect.

Code Example

```
[RequireComponent(typeof(Collider))]
public class ZoneTrigger : MonoBehaviour
{
    private IPowerZoneEffect effect;

    void Awake()
    {
        effect = GetComponent<IPowerZoneEffect>();
        GetComponent<Collider>().isTrigger = true;
    }

    void OnTriggerEnter(Collider other)
    {
        var power = other.GetComponentInParent<PlayerPower>();
        if (power != null) power.EnterZone(effect);
    }

    void OnTriggerExit(Collider other)
    {
        var power = other.GetComponentInParent<PlayerPower>();
        if (power != null) power.ExitZone(effect);
    }
}
```

```
public interface IPowerZoneEffect
{
    int GetPowerDelta();
}

public class SunZone : MonoBehaviour, IPowerZoneEffect
{
    [SerializeField] private int powerPerTick = 1;
    public int GetPowerDelta() => powerPerTick;
}

public class ShadowZone : MonoBehaviour, IPowerZoneEffect
{
    [SerializeField] private int powerPerTick = -1;
    public int GetPowerDelta() => powerPerTick;
}
```

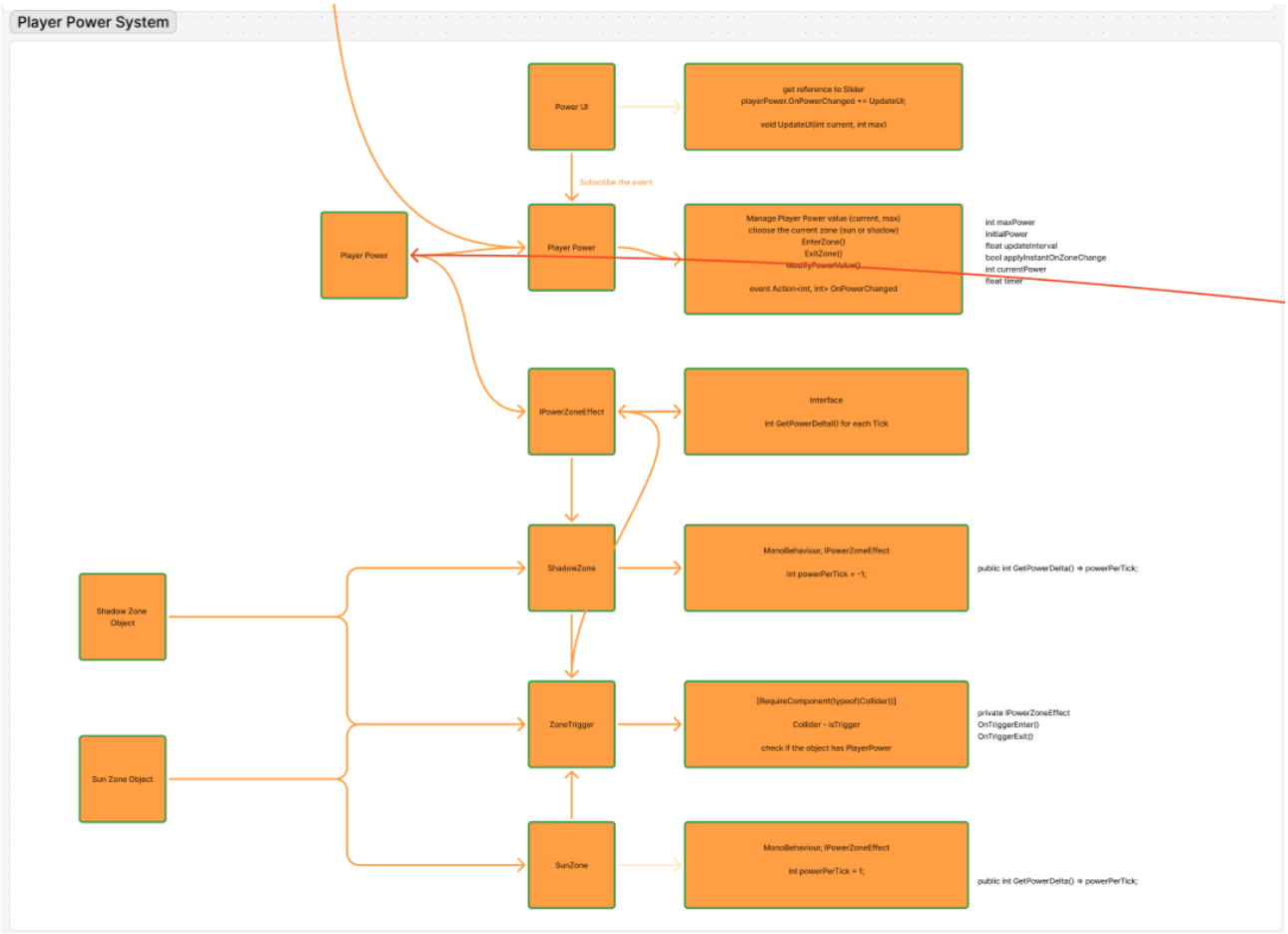
Also, to handle the conflict when multiple zone overlap, PlayerPower keeps _activeZones and selects a “current chosen zone” by strongest absolute date (tie-breaking by higher delta). That’s a clean rule centralized in controller.

```
private bool ChooseCurrentZone()
{
    IPowerZoneEffect best = null;
    int bestDeltaAbs = int.MinValue;
    int bestDelta = int.MinValue;

    foreach (var z in _activeZones)
    {
        int d = z.GetPowerDelta();
        int a = Mathf.Abs(d);
        if (a > bestDeltaAbs || (a == bestDeltaAbs && d > bestDelta))
        {
            best = z; bestDeltaAbs = a; bestDelta = d;
        }
    }

    if (!ReferenceEquals(best, _currentChosenZone))
    {
        _currentChosenZone = best;
        return true;
    }
    return false;
}
```

1.3.3 Technical Design Flowchart



1.4 Level Progression & Checkpoint System

1.4.1 General Ideas

This system be responsible for record the level progression (if passed, star and best time). Load the portal position in hub level when player is back from sub-levels. Checkpoint in the sublevel in-case player dies. Specific key to specific level. Game overall states (playing, paused). Level portal and level exit for sublevels. Player spawn point when reload the scene, portal and level UI.

1.4.2 Game Programming Patterns

Singleton

GameProgressionManager, GameManager and CheckPointSystem follow singleton pattern which will not be destroyed in next scenes. Other scripts could use GameProgressionManager.Instance or GameManager.Instance directly.

```
public class GameManager : MonoBehaviour
{
    public static GameManager Instance { get; private set; }

    private void Awake()
    {
        if (Instance != null && Instance != this)
        {
            Destroy(gameObject);
            return;
        }
        Instance = this;
        DontDestroyOnLoad(gameObject);
    }
}
```

State Pattern

In the implementation, GameManager stores a current state and changes behavior based on it (pause/resume, win/lose gating, etc.).

```
public enum GameState
{
    Playing,
    Paused,
    Win,
    GameOver
}

public class GameManager : MonoBehaviour
{
    public GameState CurrentState { get; private set; } = GameState.Playing;

    public void SetState(GameState newState)
    {
        if (CurrentState == newState) return;

        CurrentState = newState;
        OnGameStateChanged?.Invoke(CurrentState);
    }

    public void PauseGame()  => SetState(GameState.Paused);
    public void ResumeGame() => SetState(GameState.Playing);
}
```

Observer / Event Pattern

GameManager expose the event like OnGameStateChanged, OnGamePaused, OnGameResumed etc. And GameProgressManager listens to the event OnKeyCollected from KeyEvent.

```
public event System.Action<GameState> OnGameStateChanged;

public void SetState(GameState newState)
{
    if (CurrentState == newState) return;
    CurrentState = newState;

    OnGameStateChanged?.Invoke(CurrentState);
}
```

A listener (UI, portal, exit logic, etc.) can subscribe without GameManager knowing who it is:

```
private void OnEnable()
{
    if (GameManager.Instance != null)
        GameManager.Instance.OnGameStateChanged += HandleStateChanged;
}

private void OnDisable()
{
    if (GameManager.Instance != null)
        GameManager.Instance.OnGameStateChanged -= HandleStateChanged;
}

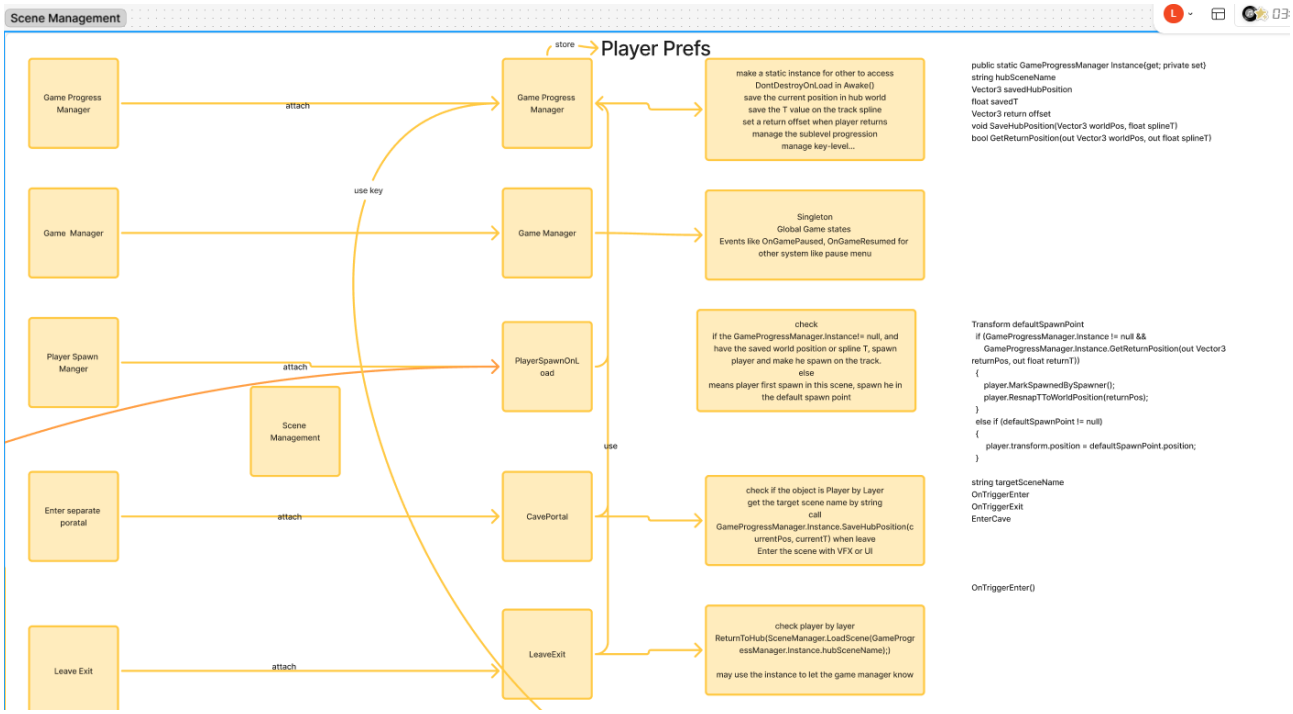
private void HandleStateChanged(GameState state)
{
    // enable/disable input, show pause menu, gate exit, etc.
}
```

Service Pattern

The Checkpoint system is implemented as a centralized service responsible for storing and providing respawn data, decoupling checkpoint detection from player spawning logic. Usage example:

```
CheckPointSystem.Instance.SetCheckpoint(position);
Vector3 spawn = CheckPointSystem.Instance.CurrentCheckpointPosition;
```

1.4.3 Technical Design Flowchart



1.5 Key & Level Unlock System

1.5.1 General Ideas

This system is designed for managing game progression through collectible keys that unlock new keys. Player has to find specific key for specific level. Keys could not be collected if the player cannot see it from the screen.

1.5.2 Game Programming Pattern

Observer / Event Pattern

The system uses a static event system (KeyEvents) to decouple key collection from UI updates and game logic.

```
public static class KeyEvents
{
    public static System.Action<string, string> OnKeyCollected;
}
```

In GameProgressionManager

```
public void CollectKey(string keyID, string unlocksLevel)
{
    if (collectedKeys.ContainsKey(keyID)) return;

    KeyData keyData = new KeyData
    {
        keyID = keyID,
        unlocksLevel = unlocksLevel,
        collectionTime = System.DateTime.Now.Ticks
    };

    collectedKeys[keyID] = keyData;


    KeyEvents.OnKeyCollected?.Invoke(keyID, unlocksLevel);


}
```

Other scripts like KeyInventoryUI

```
void Start()
{
    InitializeUI();
    KeyEvents.OnKeyCollected += OnKeyCollected;
}

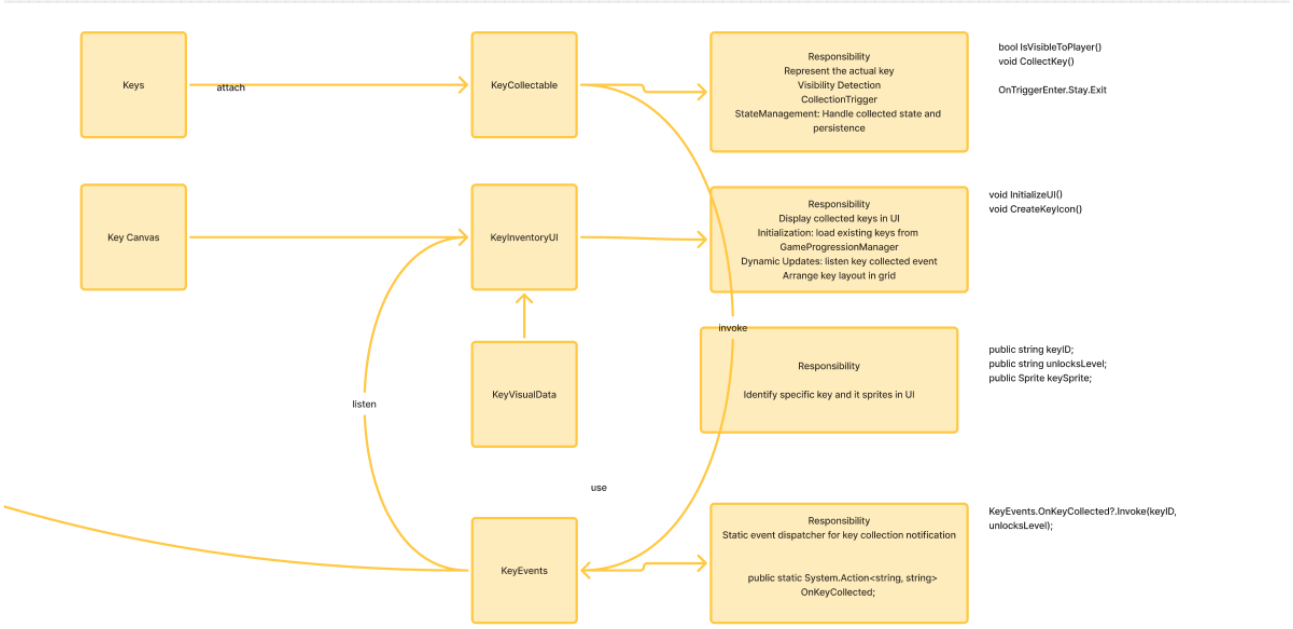
void OnDestroy()
{
    KeyEvents.OnKeyCollected -= OnKeyCollected;
}
```

Data Driven - Data Transfer Object (DTO)

KeyData will hold the required data like keyID, unlockLevel, collectionTime to encapsulate and transport data between systems. The data structure is clear, serializable for persistence and inspector visibility.

```
[System.Serializable]
public class KeyData
{
    public string keyID;
    public string unlocksLevel;
    public long collectionTime;
}
```

1.5.3 Game Design Flowchart



1.6 Weapon System

1.6.1 General Ideas

This will be the core system of the game since the game is a tool-based or weapon-based solving puzzle games. I tried to make this system to be modular, extensible and separated of concerns. This system will include weapon selection, equipment, attack behaviors and visual feedback. It supports multiple weapon types with distinct primary and secondary attack modes, connecting power consumption mechanics and UI integration.

1.6.2 Game Programming Pattern

Strategy Pattern

I used many interface to do encapsulation. For example the IWeaponAttackBehavior interface, which encapsulates different attack algorithms (foam spray and airblower) as interchangeable strategies.

In the Weapon System, different weapon attack behaviors (Air Blower, Foam Spray, Sticky Foam, Enhanced Foam) are implemented as interchangeable strategies behind a shared interface.

Code Example:

Attack behavior interface:

```
public interface IWeaponAttackBehavior
{
    void StartAttack(Transform weaponTransform, PlayerPower
playerPower);
    void UpdateAttack(Transform weaponTransform, PlayerPower
playerPower);
    void StopAttack(Transform weaponTransform, PlayerPower
playerPower);
}
```

```
public class AirBlowerAttack : IWeaponAttackBehavior
{
    public void StartAttack(Transform weaponTransform, PlayerPower playerPower)
    {
        // create air effect, start sound loop
    }

    public void UpdateAttack(Transform weaponTransform, PlayerPower playerPower)
    {
        // consume power over time
        // apply force to nearby rigidbodies
    }

    public void StopAttack(Transform weaponTransform, PlayerPower playerPower)
    {
        // stop effect and sound
    }
}
```

The weapon controller never depends on concrete attack classes

New attack types can be added without modifying existing controllers

Attack behavior can be swapped at runtime

Each attack logic follows Single Responsibility Principle

Observer and Event Pattern

I used the unity event through the weapon system, key events are OnWeaponEquipped, On-WeaponUnequipped, OnPrimaryAttackStart, OnPrimaryAttackStop, OnWeaponSelectionRe-quested,OnHotkeyPressed, and OnQuickCyclePressed. These event are listened by other scripts. To decouple component through event driven communication.

Code Example:

WeaponInputHandler raises input-related events

WeaponAttackController subscribes to attack start / stop

UI and other systems listen to weapon selection events

Repository Pattern

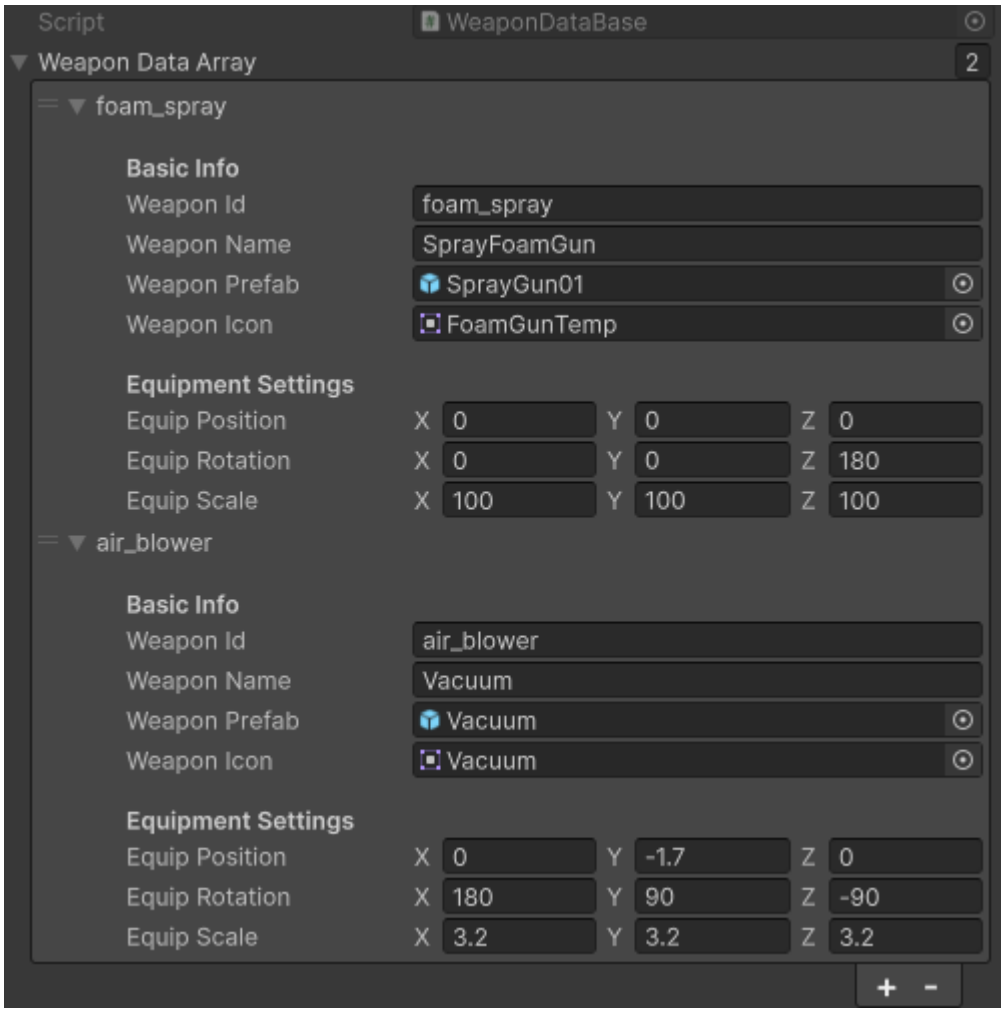
Script like WeaponDatabase (ScriptableObject) could centralizez weapon data access with caching mechanism. To make sure the system has the feature of dictionary-based weapon lookup and lazy initialization.

Code Example

```
[CreateAssetMenu(fileName = "WeaponDatabase", menuName = "Weapon System/Weapon Database")]
public class WeaponDatabase : ScriptableObject, IWeaponDataProvider
{
    [SerializeField] private WeaponData[] weaponDataArray;
}
```

```
[System.Serializable]
public class WeaponData
{
    [Header("Basic Info")]
    public string weaponId;
    public string weaponName;
    public GameObject weaponPrefab;
    public Sprite weaponIcon;

    [Header("Equipment Settings")]
    public Vector3 equipPosition = Vector3.zero;
    public Vector3 equipRotation = Vector3.zero;
    public Vector3 equipScale = Vector3.one;
}
```



Factory Pattern

WeaponAttackFactory.CreateAttackBehaviorI() centralizes creation of attack behavior instances based on AttackType enum. Weapon attack behaviors are created through a dedicated factory based on AttackType.

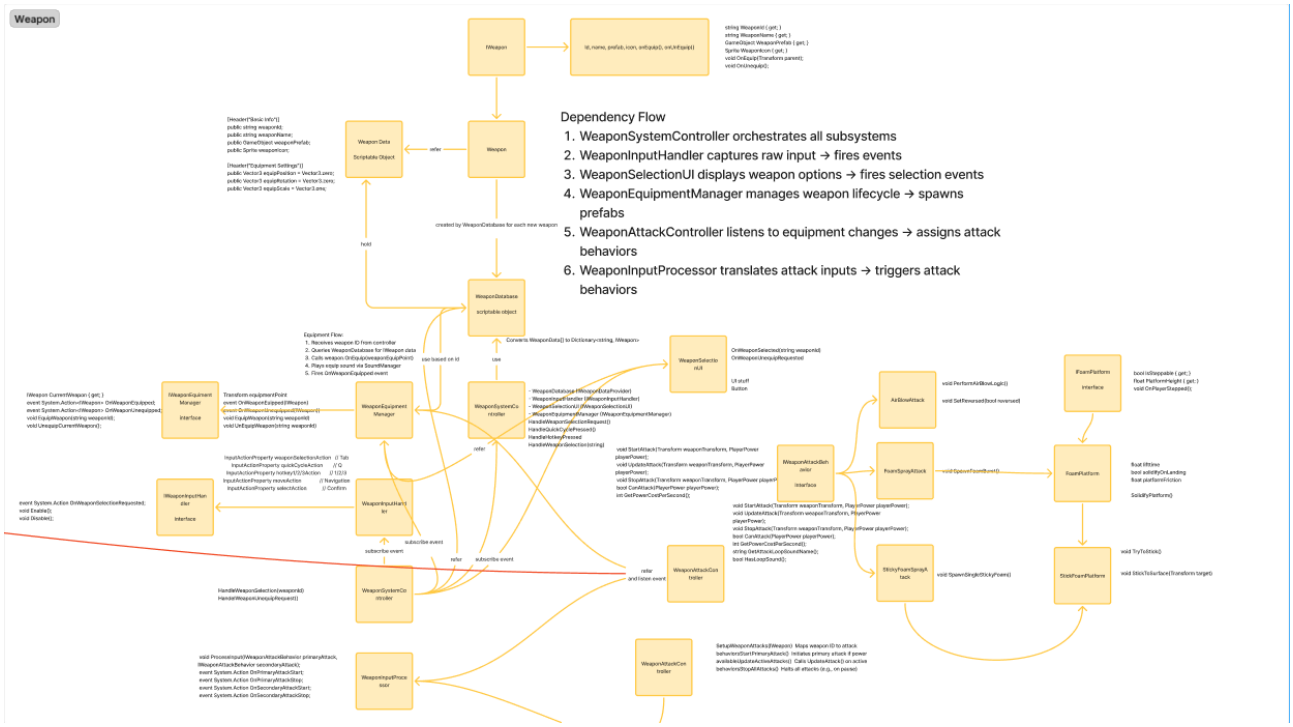
Code Example

```
public enum AttackType
{
    None,
    FoamSpray,
    AirBlower,
    // More Type
}
```

```
public static class WeaponAttackFactory
{
    public static IWeaponAttackBehavior CreateAttackBehavior(AttackType attackType)
    {
        return attackType switch
        {
            AttackType.FoamSpray => new FoamSprayAttack(),
            AttackType.AirBlower => new AirBlowerAttack(),
            AttackType.None => null,
            _ => null
        };
    }
}
```

By following this pattern, creation logic is centralized, callers depend only on IWeaponAttack-Behavior, concrete classes are hidden and supports future extension with minimal change

1.6.3 Technical Design Flowchart



1.7 NPC and Enemy State Machine System

1.7.1 General Ideas

I would like to build a solid state system for my NPCs and enemies, to make sure it is modular state machine architecture, which enables easy addition and modification. Using NavMesh-based path-finding for NPCs and enemies patrolling and chasing. For the state transitions, based on the game designer requirements, should be based on triggers and conditions. Even the general structures are the same but need to have clear distinction between animal NPCs and enemy behaviors. Use the event driven pattern to cooperate with other systems.

1.7.2 Game Programming Pattern

State Pattern

This is the core pattern to manage all AI behaviors, allowing objects to alter their behavior when internal state changes. An abstract class defines the interface, concrete state classes for each behaviors, and state transition handled by state machine.

For NPC states could be transited automatically like patrol – idle or eat – patrol. Or be changed by triggers, for example, any state could be changed to any state like hearing state by hearing trigger.

For enemy state will cooperate with player dirty clean system.

```
public abstract class State
{
    public abstract void Enter();
    public abstract void Tick(float deltaTime);
    public abstract void FixedTick(float fixedDeltaTime);
    public abstract void Exit();
}
```

```
public abstract class StateMachine : MonoBehaviour
{
    protected State currentState;

    public State CurrentState => currentState;

    public void SwitchState(State newState)
    {
        currentState?.Exit();
        currentState = newState;
        currentState?.Enter();
    }

    protected virtual void Update()
    {
        currentState?.Tick(Time.deltaTime);
    }

    protected virtual void FixedUpdate()
    {
        currentState?.FixedTick(Time.fixedDeltaTime);
    }
}
```

Concrete state example: Enemy chasing (EnemyChasingState.cs)

This shows behavior change + transitions based on conditions (detect dirty player / attack range).

```
public class EnemyChasingState : EnemyState
{
    public EnemyChasingState(EnemyStateMachine stateMachine) : base(stateMachine) { }

    public override void Tick(float deltaTime)
    {
        if (stateMachine.IsStunned) return;

        if (!stateMachine.CanDetectDirtyPlayer())
        {
            stateMachine.SwitchState(stateMachine.CreateStateFromFactory("Patrol"));
            return;
        }

        if (stateMachine.IsPlayerInAttackRange())
        {
            stateMachine.SwitchState(stateMachine.CreateStateFromFactory("Attack"));
            return;
        }

        if (stateMachine.playerTransform != null && stateMachine.NavAgent != null)
        {
            stateMachine.NavAgent.SetDestination(stateMachine.playerTransform.position);
        }
    }

    public override void Exit()
    {
        if (stateMachine.NavAgent != null && stateMachine.NavAgent.enabled)
        {
            stateMachine.NavAgent.ResetPath();
        }
    }
}
```

Concrete state example: Animal patrol (AnimalPatrolState.cs)

Shows autonomous transitions (patrol → wait → idle/eat/etc).

```
public override void Tick(float deltaTime)
{
    if (waitingAtPoint)
    {
        HandleWaiting(deltaTime);
    }
    else
    {
        HandleMovement(deltaTime);
    }
}

private void HandleWaiting(float deltaTime)
{
    waitTimer += deltaTime;

    if (animalStateMachine.Animator != null)
    {
        animalStateMachine.Animator.SetFloat(speedHash, 0f, animationDampTime, deltaTime);
    }

    if (waitTimer >= currentWaitTime)
    {
        TransitionToRandomIdleState();
    }
}
```

Factory Pattern

Factory pattern is used to create state objects dynamically based on configuration. IStateFactory interface is implemented by AnimalNPCStateMachine for animal states and EnemyStateFactory for enemy states.

```
public interface IStateFactory
{
    State CreateState(string stateName);
}
```

Enemy factory creates states by string config.

```
public class EnemyStateFactory : IStateFactory
{
    private EnemyStateMachine _stateMachine;

    public EnemyStateFactory(EnemyStateMachine stateMachine)
    {
        _stateMachine = stateMachine;
    }

    public State CreateState(string stateName)
    {
        switch (stateName.ToLower())
        {
            case "patrol":
                return new EnemyPatrolState(_stateMachine);
            case "idle":
                return new EnemyIdleState(_stateMachine);
            case "chasing":
                return new EnemyChasingState(_stateMachine);
            case "attack":
                return new EnemyAttackState(_stateMachine);
            default:
                Debug.LogWarning($"[EnemyStateFactory] Unknown state: {stateName}");
                return new EnemyIdleState(_stateMachine);
        }
    }
}
```

In AnimalNPCStateMachine, also implemente the factory creation logic.

```
public State CreateState(string stateName)
{
    switch (stateName)
    {
        case "Patrol":
            return new AnimalPatrolState(this);
        case "Idle":
            return new AnimalIdleState(this);
        case "Eating":
            return new AnimalEatingState(this);
        case "Hearing":
            return new AnimalHearingState(this);
        case "Running":
            return new AnimalRunningState(this);
        default:
            return new AnimalIdleState(this);
    }
}
```

Strategy Pattern

Different movement and behavior strategies encapsulated in separated state classes. Switching states swaps the active strategy.

For example, in AnimalRunningState, there is a independent “Flee Strategy”.

```
public override void Enter()
{
    if (animalStateMachine.Animator != null)
    {
        animalStateMachine.Animator.CrossFadeInFixedTime(runHash, crossFadeDuration);
    }

    originalSpeed = animalStateMachine.Agent.speed;
    animalStateMachine.Agent.speed = animalStateMachine.FleeSpeed;

    fleeTarget = animalStateMachine.GetNearestFleePoint();

    if (fleeTarget == null)
    {
        TransitionToSafeState();
    }
}

public override void Tick(float deltaTime)
{
    if (fleeTarget == null)
    {
        TransitionToSafeState();
        return;
    }

    MoveToPosition(fleeTarget.position, deltaTime);

    Vector3 velocity = animalStateMachine.Agent.desiredVelocity;
    if (velocity.sqrMagnitude > 0.01f)
    {
        FaceDirection(velocity);
    }

    if (CheckArrival())
    {
        TransitionToSafeState();
    }
}
```

Observer / Event Pattern

Using Unity event system and trigger mechanisms for state transitions.

Unity trigger： broadcast to many targets (Observer-style)

StateTransitionTrigger.cs + StateTransitionConfig.cs + ITriggerable.cs

Subscriber interface:

```
public interface ITriggerable
{
    Transform Transform { get; }
    void OnTriggered(StateTransitionConfig config);
    bool CanBeTriggered(StateTransitionConfig config);
}
```

Publisher trigger: StateTransitionTrigger.cs

```
private void OnTriggerEnter(Collider other)
{
    if (Time.time - lastTriggerTime < config.cooldownTime)
        return;

    if (hasTriggered && config.triggerOnce)
        return;

    if (!other.CompareTag("Player"))
        return;

    TriggerStateTransition();
}

private void TriggerStateTransition()
{
    Collider[] colliders = Physics.OverlapSphere(transform.position, config.affectRadius);

    foreach (Collider col in colliders)
    {
        ITriggerable triggerable = col.GetComponent<ITriggerable>();
        if (triggerable == null) continue;

        if (!config.IsTargetValid(triggerable)) continue;
        if (!triggerable.CanBeTriggered(config)) continue;

        triggerable.OnTriggered(config);
    }

    hasTriggered = true;
    lastTriggerTime = Time.time;

    Debug.Log($"[StateTransitionTrigger] Triggered: {config.targetStateName}");
}
```

A triggerable subscriber reacts by switching state: AnimalNPCStateMachine.

```
public override void OnTriggered(StateTransitionConfig config)
{
    if (!CanBeTriggered(config))
    {
        return;
    }

    State newState = CreateState(config.targetStateName);
    if (newState != null)
    {
        SwitchState(newState);
    }
}
```

C# event subscription (classic Observer) — Enemy listens to Player Dirt system

From EnemyStateMachine.cs

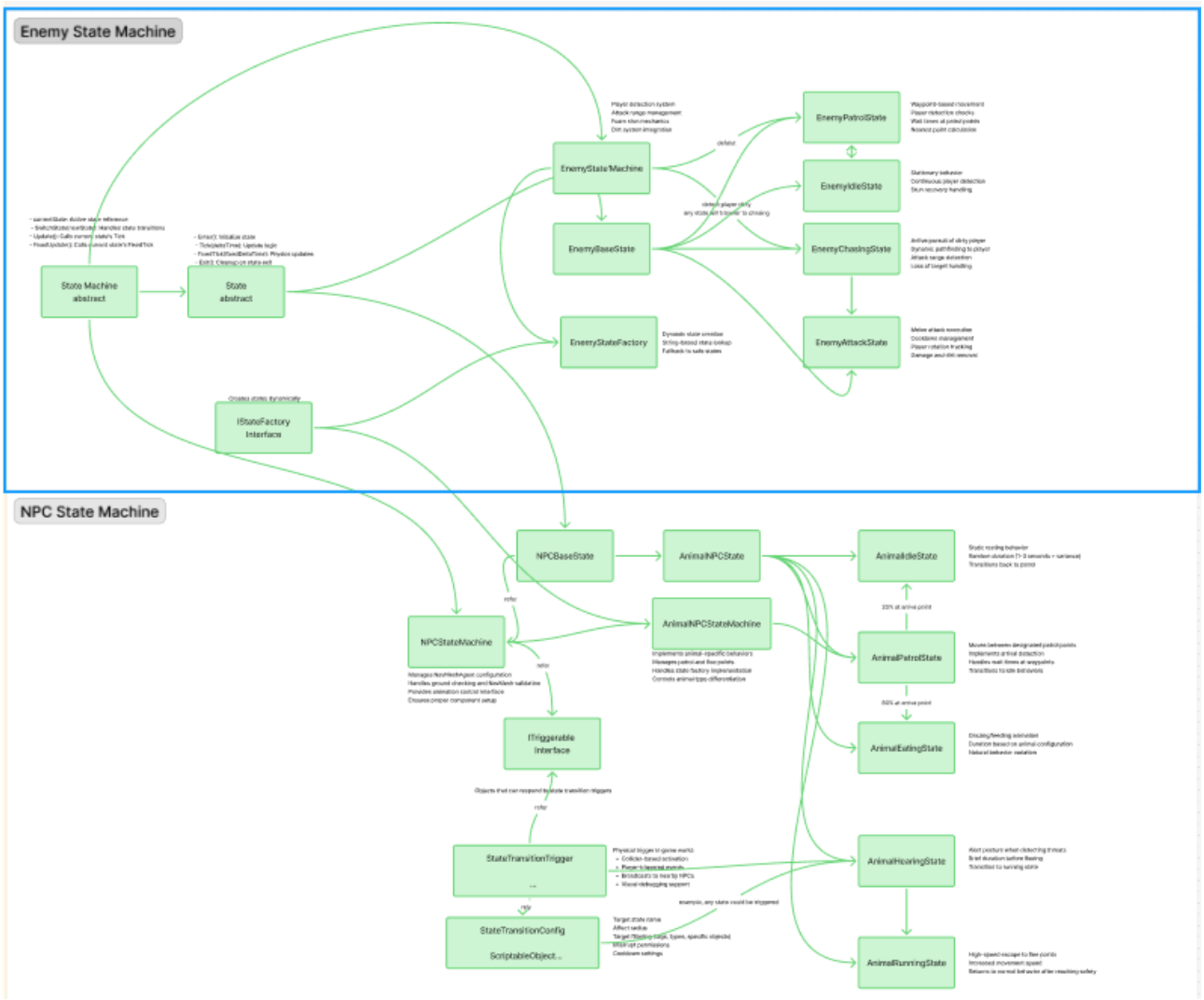
```
private void FindPlayer()
{
    var player = GameObject.FindGameObjectWithTag("Player");
    if (player != null)
    {
        playerTransform = player.transform;
        playerDirtSystem = player.GetComponent<PlayerDirtSystem>();

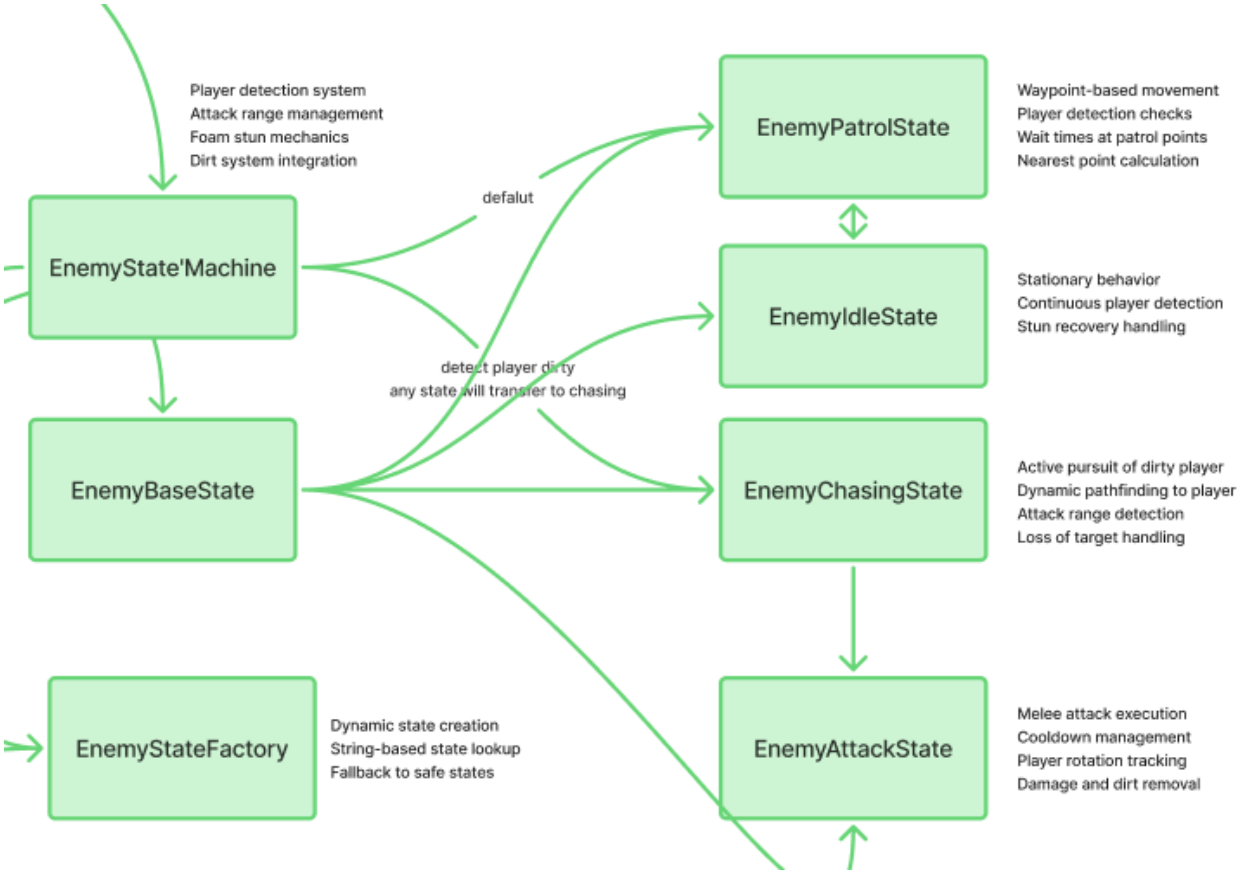
        if (playerDirtSystem != null)
        {
            playerDirtSystem.OnBecameClean += OnPlayerBecameClean;
        }
    }
}

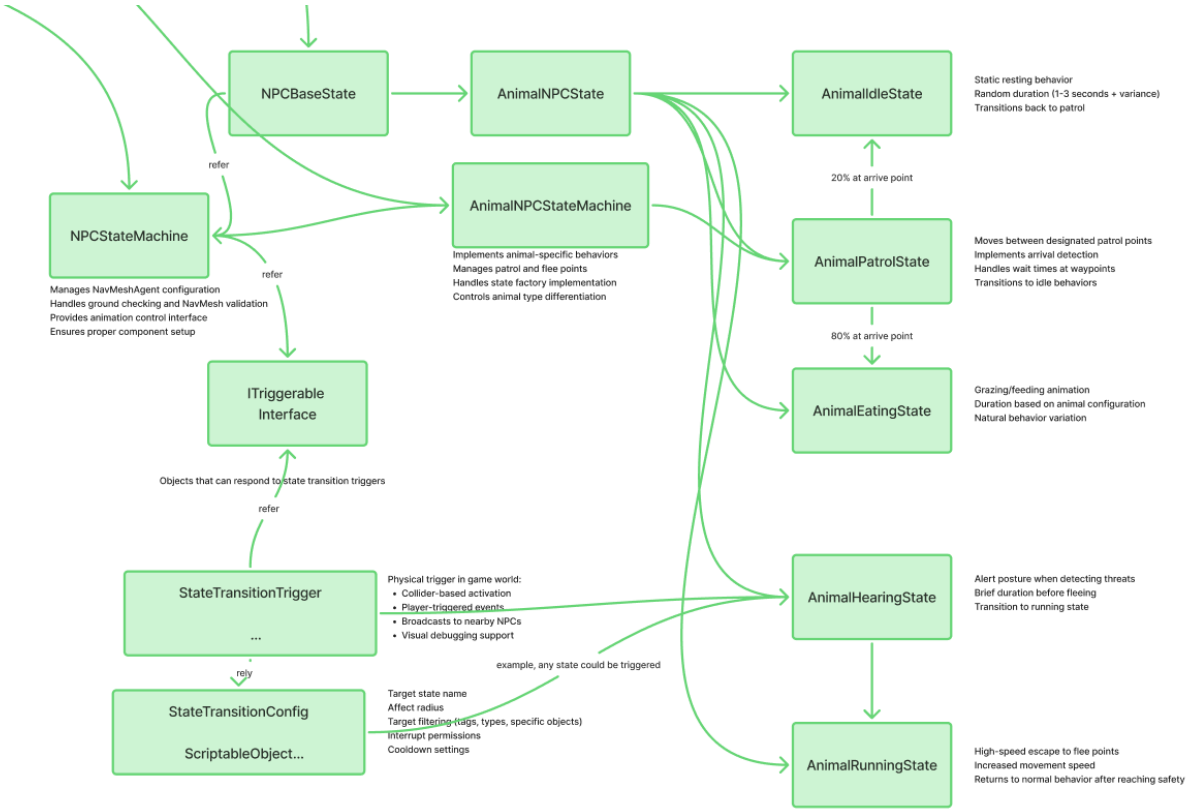
private void OnPlayerBecameClean()
{
    if (currentState is EnemyChasingState || currentState is EnemyAttackState)
    {
        SwitchState(_stateFactory.CreateState("Patrol"));
    }
}

void OnDestroy()
{
    if (playerDirtSystem != null)
    {
        playerDirtSystem.OnBecameClean -= OnPlayerBecameClean;
    }
}
```

1.7.3 Technical Design Flowchart




Player detection system
Attack range management
Foam stun mechanics
Dirt system integration
EnemyState'Machine
defalut
detect player dirty
any state will transfer to chasing
EnemyPatrolState
Waypoint-based movement
Player detection checks
Wait times at patrol points
Nearest point calculation
EnemyIdleState
Stationary behavior
Continuous player detection
Stun recovery handling
EnemyBaseState
EnemyChasingState
Active pursuit of dirty player
Dynamic pathfinding to player
Attack range detection
Loss of target handling
EnemyStateFactory
Dynamic state creation
String-based state lookup
Fallback to safe states
EnemyAttackState
Melee attack execution
Cooldown management
Player rotation tracking
Damage and dirt removal


NPCBaseState
AnimalNPCState
AnimalIdleState
Static resting behavior
Random duration (1-3 seconds + variance)
Transitions back to patrol
refer
20% at arrive point
AnimalNPCStateMachine
Implements animal-specific behaviors
Manages patrol and flee points
Handles state factory implementation
Controls animal type differentiation
NPCStateMachine
Manages NavMeshAgent configuration
Handles ground checking and NavMesh validation
Provides animation control interface
Ensures proper component setup
refer
AnimalPatrolState
Moves between designated patrol points
Implements arrival detection
Handles wait times at waypoints
Transitions to idle behaviors
80% at arrive point
ITriggerable Interface
Objects that can respond to state transition triggers
refer
AnimalEatingState
Grazing/feeding animation
Duration based on animal configuration
Natural behavior variation
StateTransitionTrigger
...
Physical trigger in game world:
• Collider-based activation
• Player-triggered events
• Broadcasts to nearby NPCs
• Visual debugging support
AnimalHearingState
Alert posture when detecting threats
Brief duration before fleeing
Transition to running state
rely
example, any state could be triggered
StateTransitionConfig
ScriptableObject...
Target state name
Affect radius
Target filtering (tags, types, specific objects)
Interrupt permissions
Cooldown settings
AnimalRunningState
High-speed escape to flee points
Increased movement speed
Returns to normal behavior after reaching safety

1.8 Reaction Panel

1.8.1 General Ideas

The general idea for this player reaction panel system is that using the world space UI panel to show different visual sprites by colliding different triggers. Sprites showing will be based on the reaction key string.

1.8.2 Service Oriented Architecture

Service layer: ReactionManger is the singleton service managing all reaction. IReactionService provides the interface for decoupled access.

Data Layer: ReactionLibrary is the scriptable object storing sprite mappings. ReactionOptions is the configuration struct for reaction behavior

Presentation Layer: ReactionBubblePresenter is the visual display component

Controller Layer: ProximityReactionEmitter and PowerReactionController controls the detail sprite showing.

1.8.3 Game Programming Pattern

Service Locator Pattern

ReactionManager as the singleton service, exposes a accessible Instance (typed as IReaction-Service) could have benefits of global access to reaction system. Also follow the simplified dependency management feature. So callers don't need a direct reference to the manager (no scene wiring required). Call sites depend on the abstraction (IReactionService), not a concrete class. Global availability fits a UI feedback system used by many unrelated gameplay scripts.

```
public class ReactionManager : MonoBehaviour, IReactionService
{
    public static IReactionService Instance { get; private set; }

    private void Awake()
    {
        if (Instance != null && Instance != this)
        {
            Destroy(gameObject);
            return;
        }
        Instance = this;
        DontDestroyOnLoad(gameObject);
    }
}
```

Data-Driven Design

ReactionLibrary as the sprite repository. Sprites are not hardcoded inside emitters or the presenter. Instead, a key string is resolved through a ScriptableObject library.

```
[CreateAssetMenu(menuName = "Reactions/Reaction Library")]
public class ReactionLibrary : ScriptableObject, IReactionLibrary
{
    [System.Serializable]
    public class Entry { public string key; public Sprite sprite; }

    [SerializeField] private List<Entry> entries = new();
    private Dictionary<string, Sprite> _dict;

    void OnEnable()
    {
        _dict = new Dictionary<string, Sprite>();
        foreach (var e in entries)
            if (!string.IsNullOrWhiteSpace(e.key) && e.sprite != null)
                _dict[e.key] = e.sprite;
    }

    public bool TryGet(string key, out Sprite sprite)
    {
        if (_dict == null) OnEnable();
        return _dict.TryGetValue(key, out sprite);
    }
}
```

Presentation Pattern

ReactionBubblePresenter is the dedicated visual component handling following the anchor, facing the camera, priority and hide / show time.

```
public void ShowReaction(Sprite sprite, in ReactionOptions options)
{
    if (sprite == null) return;

    if (IsBusy && options.overrideIfLowerPriority == false && options.priority < CurrentPriority)
        return;

    CurrentPriority = options.priority;

    if (_showRoutine != null) StopCoroutine(_showRoutine);
    _showRoutine = StartCoroutine(CoShow(sprite, options.duration <= 0 ? 1.5f : options.duration));
}
```

Observer Pattern

Listens the event like playerPower.OnPowerChanged to show or hide reactions through the reaction service, without touching UI details.

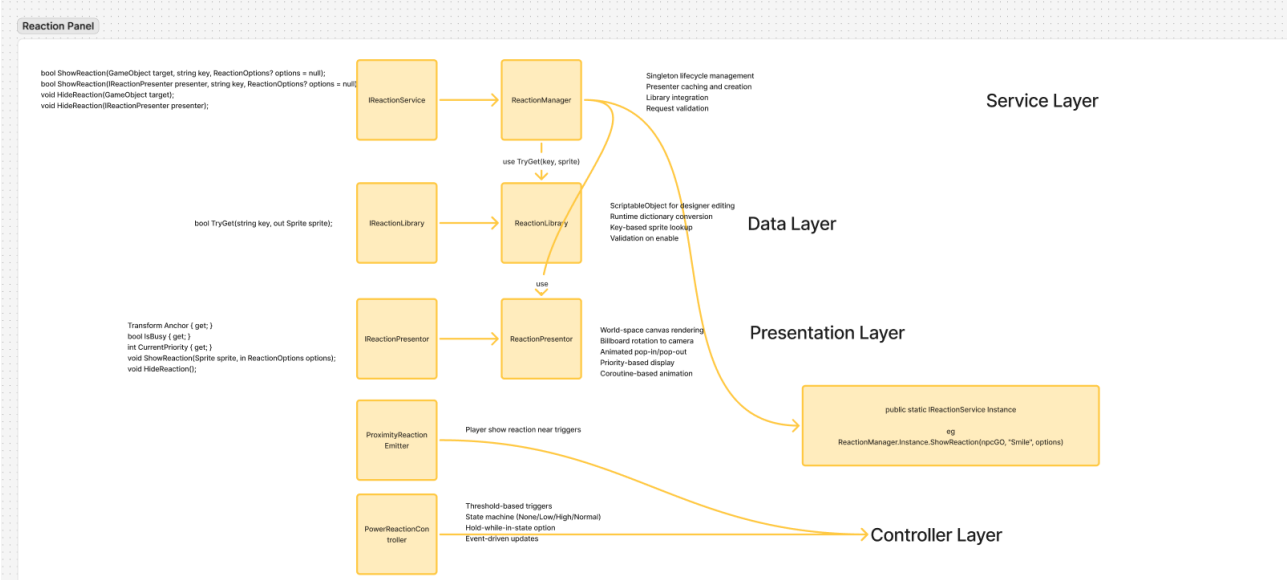
```
void OnEnable()
{
    playerPower.OnPowerChanged += OnPowerChanged;
    EvaluateImmediate(playerPower.Current, playerPower.Max);
}

void OnDisable()
{
    playerPower.OnPowerChanged -= OnPowerChanged;
}

private void Show(string key)
{
    var svc = ReactionManager.Instance;
    if (svc == null) return;

    var opts = new ReactionOptions { duration = showDuration, priority = priority };
    svc.ShowReaction(gameObject, key, opts);
}
```

1.8.4 Technical Design Flowchart



1.9 Player Health System

1.9.1 General Ideas

I would like to make a completed player health system which could deal with player health, damage, healing and respawn mechanics. The whole system is interface based with event driven. The whole system will also integrate the UI presentation and connect with check point system.

1.9.2 Game Programming Pattern

Interface Segregation

The system will use multiple separated interfaces for different responsibilities.

For example, this system will include IHealth, IHealthEvents, IHealthUI, IDamageDealer and IHealthRestore. UI code depends only on IHealthUI, not on damage/heal logic. Traps depend only on IDamageDealer, not on UI or respawn rules. Pickups depend only on IHealthRestore + IHealth, not on trap logic.

```
public interface IHealth
{
    int MaxHealth { get; }
    int CurrentHealth { get; }
    bool IsDead { get; }

    bool TakeDamage(int damage);
    bool Heal(int amount);
    void ResetToMaxHealth();
}
```

```
public interface IHealthEvents
{
    System.Action<int, int> OnHealthChanged { get; set; }
    System.Action OnDeath { get; set; }
    System.Action<int> OnDamage { get; set; }
    System.Action<int> OnHeal { get; set; }
}
```

```
public interface IDamageDealer
{
    int DamageAmount { get; }
    void DealDamage(IHealth target);
}

public interface IHealthRestore
{
    int RestoreAmount { get; }
    void RestoreHealth(IHealth target);
}
```

```
public interface IHealthUI
{
    void UpdateHealthDisplay(int currentHealth, int maxHealth);
    void PlayDamageEffect(int damageAmount);
    void PlayHealEffect(int healAmount);
}
```

Observer / Event Pattern

HealthSystem is the event publisher. PlayerHealth wires subscribers (UI + death handler), and other systems respond without tight coupling—matching the design doc's event-driven claim. Event based communication between scripts, like OnHealthChanged, OnDeath, OnDamage and OnHeal.

```
public class HealthSystem : IHealth, IHealthEvents
{
    public System.Action<int, int> OnHealthChanged { get; set; }
    public System.Action OnDeath { get; set; }
    public System.Action<int> OnDamage { get; set; }
    public System.Action<int> OnHeal { get; set; }

    public bool TakeDamage(int damage)
    {
        if (damage <= 0 || IsDead) return false;

        int previousHealth = currentHealth;
        currentHealth = Mathf.Max(0, currentHealth - damage);

        OnDamage?.Invoke(damage);
        OnHealthChanged?.Invoke(currentHealth, maxHealth);

        if (currentHealth <= 0 && previousHealth > 0)
            OnDeath?.Invoke();

        return true;
    }

    public bool Heal(int amount)
    {
        if (amount <= 0 || currentHealth >= maxHealth) return false;

        int previousHealth = currentHealth;
        currentHealth = Mathf.Min(maxHealth, currentHealth + amount);
        int actualHealAmount = currentHealth - previousHealth;

        if (actualHealAmount > 0)
        {
            OnHeal?.Invoke(actualHealAmount);
            OnHealthChanged?.Invoke(currentHealth, maxHealth);
            return true;
        }
        return false;
    }
}
```

PlayerHealth subscribes UI methods to health events (and also listens to death to trigger respawn logic).

```
healthSystem = new HealthSystem(maxHealth);
healthSystem.OnDeath += HandleDeath;

healthSystem.OnHealthChanged += healthUI.UpdateHealthDisplay;
healthSystem.OnDamage += healthUI.PlayDamageEffect;
healthSystem.OnHeal += healthUI.PlayHealEffect;
```

Healing pickup triggers health via interface, not concrete UI

```
public class HealthPickup : MonoBehaviour, IHealthRestore
{
    public void RestoreHealth(IHealth target)
    {
        if (target.CurrentHealth >= target.MaxHealth) return;

        bool restored = target.Heal(restoreAmount);
        if (restored) StartCoroutine(CollectAnimation());
    }
}
```

Template Pattern

BaseTrap as the abstract class will get inherited from HeavyTrap, MediumTrap, LightTrap, DelayedTrap and MovingTrap.

Template Method in BaseTrap (invariant structure)

DealDamage() is the template: it enforces invulnerability timing, applies damage, then runs hook methods.

```
public virtual void DealDamage(IHealth target)
{
    if (hasInvulnerabilityPeriod && Time.time - lastDamageTime < invulnerabilityDuration)
        return;

    if (target.TakeDamage(damageAmount))
    {
        lastDamageTime = Time.time;

        OnDamageDealt();     // hook
        PlayHitEffect();     // hook

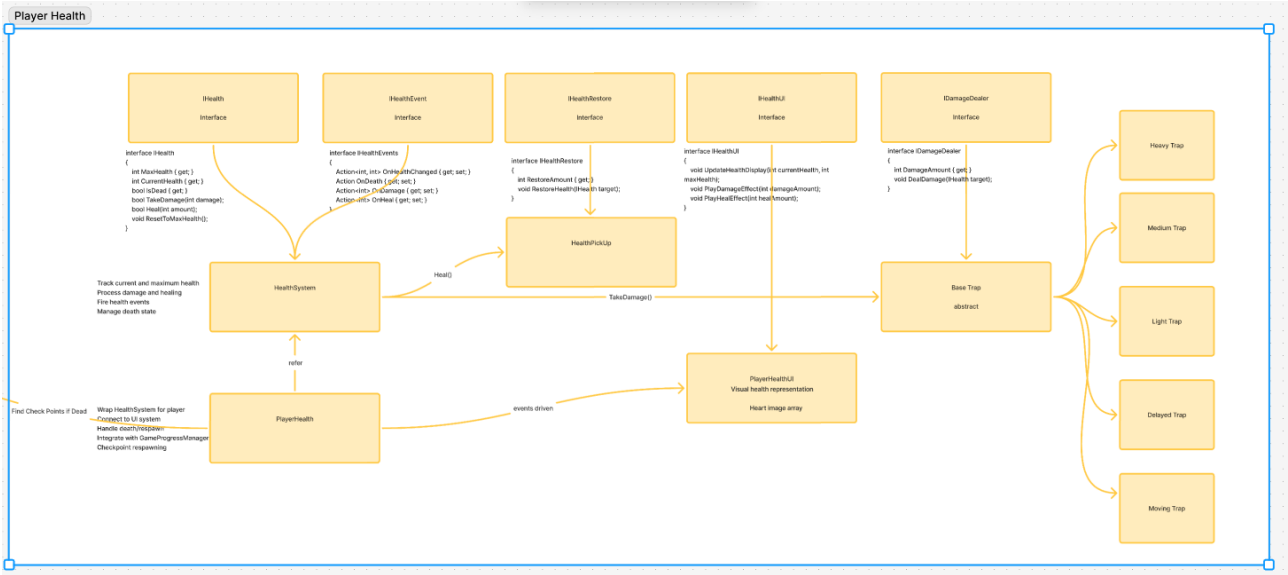
        if (destroyAfterHit)
            Destroy(gameObject);
    }
}
```

```
public class LightTrap : BaseTrap
{
    void Awake()
    {
        damageAmount = 1;
        hitEffect.gameObject.SetActive(false);
    }
}

public class MediumTrap : BaseTrap
{
    void Awake() => damageAmount = 2;
}

public class HeavyTrap : BaseTrap
{
    void Awake() => damageAmount = 3;
}
```

1.9.3 Technical Design Flowchart



1.10 Player Dirty and Clean System

1.10.1 General Ideas

I will use a dedicated data structure BodyPart to collect the separated body part of the player and its original material and the dirty material. Then a PlayerDirtySystem to manage all the dirty and clean functions and set the events for other scripts to listen. DirtyZones and CleanZones will call the required method in PlayerDirtySystem when collision happens.

1.10.2 Game Programming Pattern

Component Pattern

PlayerDirtySystem and BodyParts will be attached as component on player and CleanZone and DirtyZone will be attached to the respective zones. So the scripts' logic will only be responsible for its own object.

For example, in DirtZone:

```
private void OnTriggerStay(Collider other)
{
    if (!other.CompareTag("Player")) return;

    var dirtSystem = other.GetComponent<PlayerDirtSystem>();
    if (!dirtSystem) return;

    float delta = dirtPerSecond * Time.deltaTime;

    if (dirtyAllParts)
        dirtSystem.AddDirtToAll(delta);
    else
        dirtSystem.AddDirtToRandom(partsToDirt, delta);
}
```

Observer Pattern

Dirt / clean events boardcast to other systems. Events like OnDirtChanged, OnBodyPartDirtied and OnBodyPartCleaned will be listened by other system like the EnemyStateMachie, EnemyAttack-State and EnemyChasingState.

In PlayerDirtSystem publish the events:

```
public System.Action<int, int> OnDirtChanged;
public System.Action OnBecameDirty;
public System.Action OnBecameClean;
public System.Action<BodyPart> OnBodyPartDirtied;
public System.Action<BodyPart> OnBodyPartCleaned;

public bool IsAnyDirty => _dirtyPartsCount > 0;
public bool IsFullyClean => _dirtyPartsCount == 0;
```

Events are raised at the exact state transition moments:

```
if (before <= 0f && part.dirtLevel > 0f)
{
    part.isDirty = true;
    _dirtyPartsCount++;
    OnBodyPartDirtied?.Invoke(part);
    OnDirtChanged?.Invoke(_dirtyPartsCount, bodyParts.Count);
}

if (before > 0f && part.dirtLevel <= 0f)
{
    part.isDirty = false;
    _dirtyPartsCount = Mathf.Max(0, _dirtyPartsCount - 1);
    OnBodyPartCleaned?.Invoke(part);
    OnDirtChanged?.Invoke(_dirtyPartsCount, bodyParts.Count);
}
```

```
if (wasAnyDirty && IsFullyClean)
    OnBecameClean?.Invoke();
```

Other scripts like Enemy AI will subscribe the events

```
playerDirtSystem = player.GetComponent<PlayerDirtSystem>();
if (playerDirtSystem != null)
{
    playerDirtSystem.OnBecameClean += OnPlayerBecameClean;
}
```

```
private void OnPlayerBecameClean()
{
    if (currentState is EnemyChasingState || currentState is EnemyAttackState)
        SwitchState(_stateFactory.CreateState("Patrol"));
}
```

Data Pattern

BodyPart will act as the collection of the body part name, render, material and index. PlayerDirtySystem will refer it as a list. So it follows single responsibility policy.

```
[System.Serializable]
public class BodyPart
{
    public string partName = "Body Part";
    public Renderer renderer;

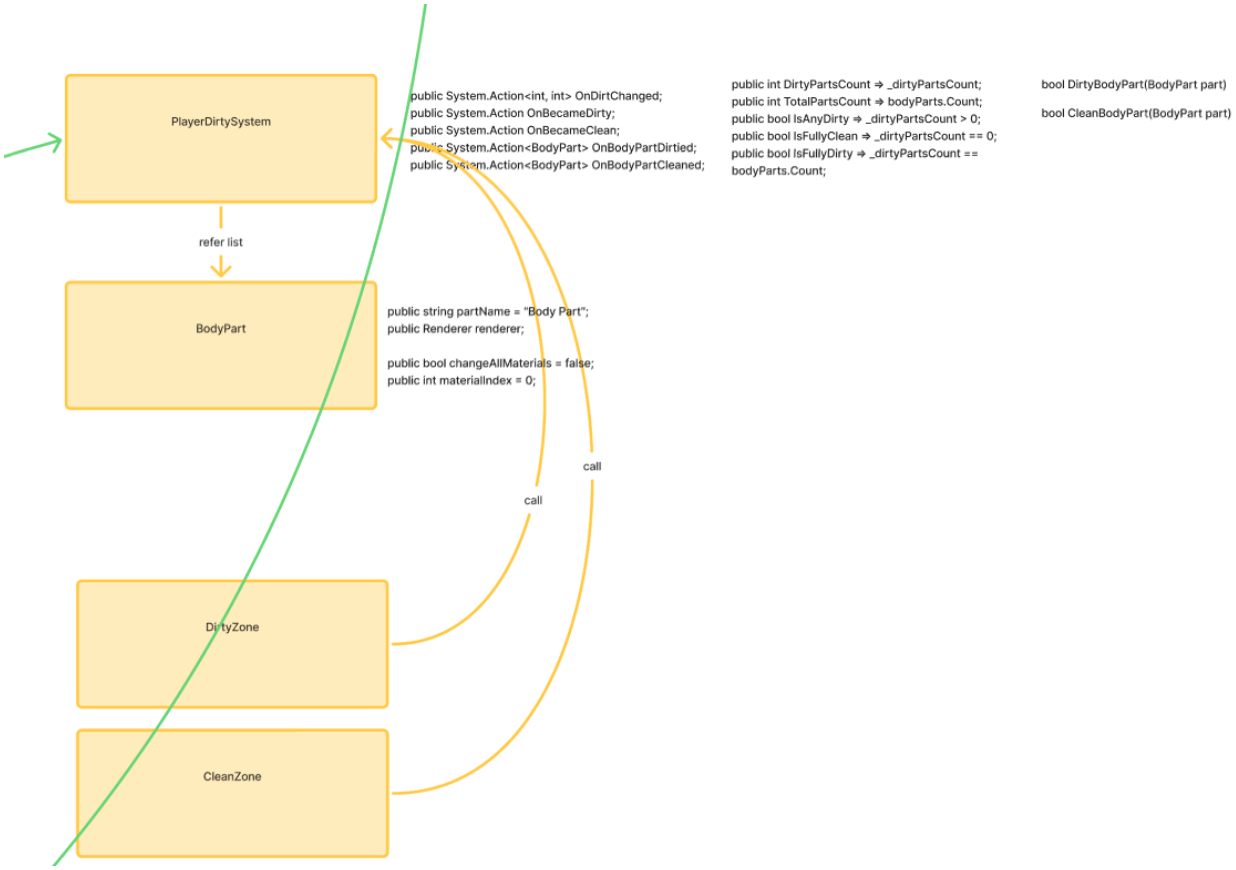
    public bool changeAllMaterials = false;
    public int materialIndex = 0;

    [HideInInspector] public Material originalMaterial;
    [HideInInspector] public Material[] originalMaterials;

    [HideInInspector] public bool isDirty = false;
    [HideInInspector] public float dirtLevel = 0f;

    [HideInInspector] public MaterialPropertyBlock mpb;
    [HideInInspector] public Color baseColor = Color.white;
    [HideInInspector] public float baseMetallic = 0f;
    [HideInInspector] public float baseSmoothness = 0.5f;
}
```

1.10.3 Technical Design Flowchart



1.11 Slippery and Water Zone

1.11.1 General Ideas

The player main movement component SplineRunnerRB will be limited in the special zones (movement speed and jumping force), at the same time, the system will check with the WeaponEquipmentManager see if the airblower weapon is equipped. Then enable air-flow movement, movement will subscribing the attacking input event. Once the player is out of the zones, movement modifiers are reverted , air-flow movement is disabled and input events are unsubscribed.

1.11.2 Game Programming Pattern

Component Pattern

Each functionality (zone detection, slippery effect, air flow movement) is isolated as a separate component attached to objects. They interact through component references instead of hard coupling.

For example, Zone detteion component: SlipperyZoneController.cs, WaterZoneController.cs. Both zones are responsible only for detecting player entry/exit and notifying zone effects.

```
private void OnTriggerEnter(Collider other)
{
    if (!other.CompareTag("Player")) return;

    var zoneEffect = other.GetComponent<IZoneEffect>();
    if (zoneEffect != null)
    {
        zoneEffect.OnZoneEnter(this);
    }
}

private void OnTriggerExit(Collider other)
{
    if (!other.CompareTag("Player")) return;

    var zoneEffect = other.GetComponent<IZoneEffect>();
    if (zoneEffect != null)
    {
        zoneEffect.OnZoneExit(this);
    }
}
```

Zone effect component: SlipperyZoneEffect.cs, WaterZoneEffect.cs. Each zone effect coordinates which behaviors should be enabled or disabled.

```
public void OnZoneEnter(WaterZoneController zone)
{
    movementModifier.ApplyModifier();
    airFlowController.TryEnableAirFlow();
}

public void OnZoneExit(WaterZoneController zone)
{
    movementModifier.RemoveModifier();
    airFlowController.DisableAirFlow();
}
```

Strategy Pattern

Interfaces like IZoneEffect and IAirFlowMovement define interchangeable behaviors. Different types of zones or movement effects can replace current implementation without modifying the controller.

Zone Effect Strategy

```
public interface IZoneEffect
{
    void OnZoneEnter(Component zone);
    void OnZoneExit(Component zone);
}
```

Movement Modifier Strategy:

```
public interface IMovementModifier
{
    void ApplyModifier();
    void RemoveModifier();
}
```

Air Flow Movement Strategy

```
public interface IAirFlowMovement
{
    void Enable();
    void Disable();
}
```

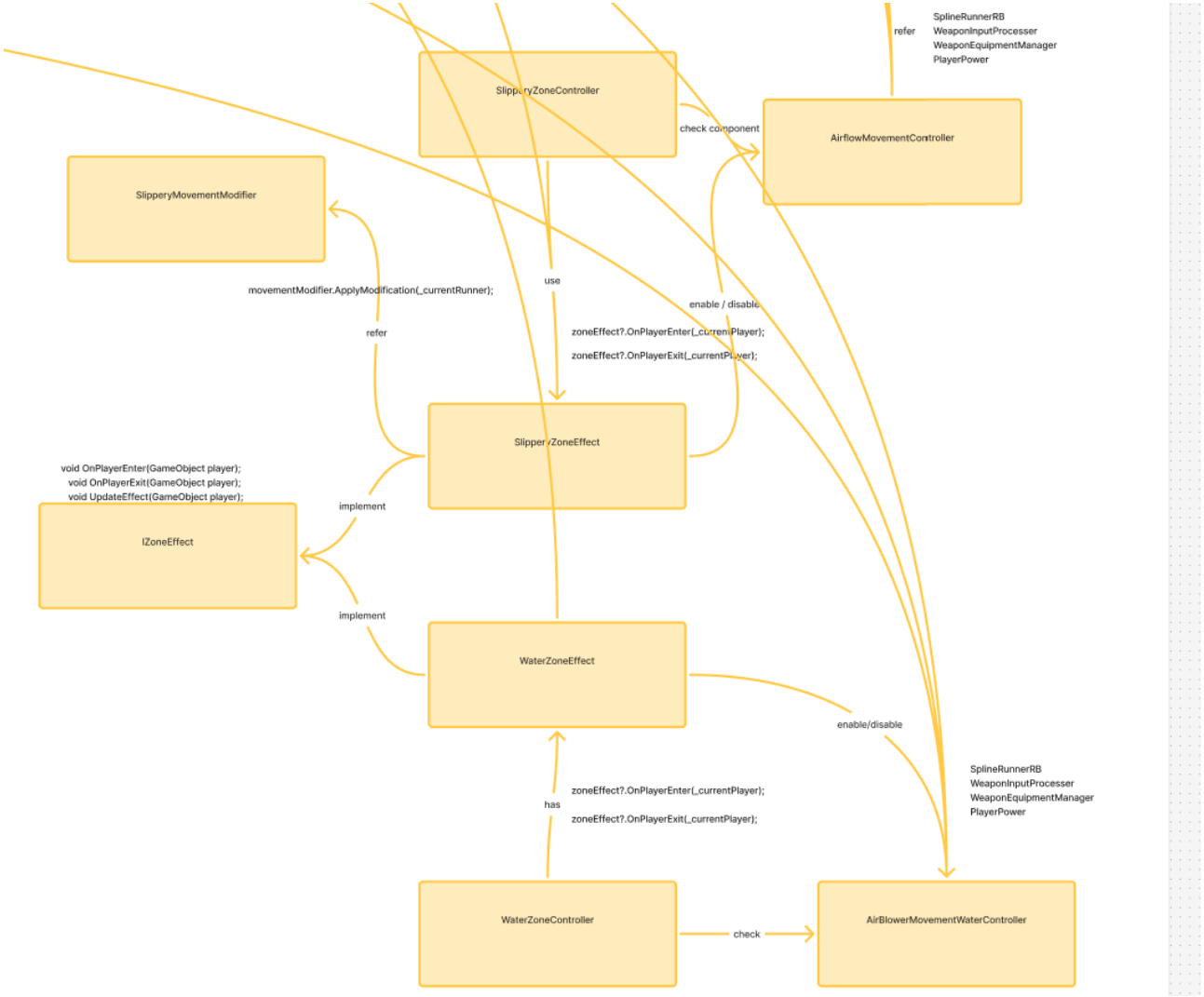
Implementation:

```
public class AirFlowMovementController
    : MonoBehaviour, IAirFlowMovement
{
    // airflow behavior
}
```

Observer Pattern

When entering the zone, the controller subscribes to player input events (OnPrimaryAttackStart, etc.), and unsubscribes on exit. This is exactly how Observer Pattern works.

1.11.3 Technical Design Flowchart



1.12 Setting Controls

1.12.1 General Ideas

Settings like BGM volume, SFX volume and Brightness could be changed via slider in both Main Menu Scene and Game Scene (Pause Menu). The values will be stored and applied for all scenes. The global instance will be used for GolbalSettingManager, BrightnessManager, SoundManager and BGMSceneManager. A scriptableObject sound library will be used to store the sound string names and its responsible actual sound file. UI like setting panels, sliders, button will be integrated.

1.12.2 Game Programming Pattern

Singleton Pattern

GlobalSettingManager, BrightnessManager, SoundManager and BGMSceneManger will not be destroyed between scenes. Instance will be used and called from other scripts.

GlobalSettingsManager persists and exposes a global instance

```
public static GlobalSettingsManager Instance { get; private set; }

private void Awake()
{
    if (Instance != null && Instance != this)
    {
        Destroy(gameObject);
        return;
    }

    Instance = this;
    DontDestroyOnLoad(gameObject);

    LoadAllSettings();
}
```

BrightnessManager and BGMSceneManger :

```
public static BrightnessManager Instance { get; private set; }

private void Awake()
{
    if (Instance != null && Instance != this)
    {
        Destroy(gameObject);
        return;
    }
    Instance = this;
    DontDestroyOnLoad(gameObject);
}
```

```
public static BGMSceneManager Instance { get; private set; }

private void Awake()
{
    if (Instance != null && Instance != this)
    {
        Destroy(gameObject);
        return;
    }

    Instance = this;
    DontDestroyOnLoad(gameObject);
}
```

SoundMnager is also a singleton like (typed as interface)

```
public static ISoundPlayer Instance { get; private set; }

private void Awake()
{
    if (Instance != null && Instance != this as ISoundPlayer)
    {
        Destroy(gameObject);
        return;
    }

    Instance = this;
    DontDestroyOnLoad(gameObject);
}
```

Observer Pattern

Events like OnSFXChanged, OnBGMVolumeChanged, OnBrightnessChanged will be listened by scripts like BGMVolumeSetting, SFXVolumeSetting and BrightnessSetting.

Publisher: events declared and invoked in GlobalSettingManager

```
public event Action<float> OnSFXVolumeChanged;
public event Action<float> OnBGMVolumeChanged;
public event Action<float> OnBrightnessChanged;
public void SetSFXVolume(float volume)

{
    _sfxVolume = Mathf.Clamp01(volume);
    PlayerPrefs.SetFloat(SFX_VOLUME_KEY, _sfxVolume);
    PlayerPrefs.Save();

    if (SoundManager.Instance != null)
    {
        SoundManager.Instance.SetVolume(_sfxVolume);
    }

    OnSFXVolumeChanged?.Invoke(_sfxVolume);
}
```

Subscriber: SFXVolumeSetting Listens and updated UI

```
protected override void SubscribeToGlobalEvents()
{
    if (GlobalSettingsManager.Instance != null)
    {
        GlobalSettingsManager.Instance.OnSFXVolumeChanged += OnGlobalSFXVolumeChanged;
    }
}

private void OnGlobalSFXVolumeChanged(float value)
{
    UpdateSliderSilently(value);
}
```

Same structure exists for:

BGMVolumeSetting - OnBGMVolumeChanged

BrightnessSetting – OnBrightnessChanged

Template Pattern

SettingController will define the basic things like loading value, listen event like OnValueChanged from slidder, change and apply the slider value and save the new value to PlayerPrefs.

Key template flow:

```
protected virtual void Start()
{
    LoadSetting();
    _isInitialized = true;
}

protected virtual void OnEnable()
{
    settingSlider.onValueChanged.AddListener(OnValueChanged);
}

protected virtual void OnValueChanged(float value)
{
    ApplySetting(value);
    PlayerPrefs.SetFloat(settingKey, value);
    UpdateValueText(value);
}

protected abstract void ApplySetting(float value);
```

Derived classes only implement the variable step: ApplySetting

Example (BrightnessSetting):

```
protected override void ApplySetting(float value)
{
    if (GlobalSettingsManager.Instance != null)
    {
        GlobalSettingsManager.Instance.SetBrightness(value);
    }
    else if (BrightnessManager.Instance != null)
    {
        BrightnessManager.Instance.SetBrightness(value);
    }
}
```

Data Driven

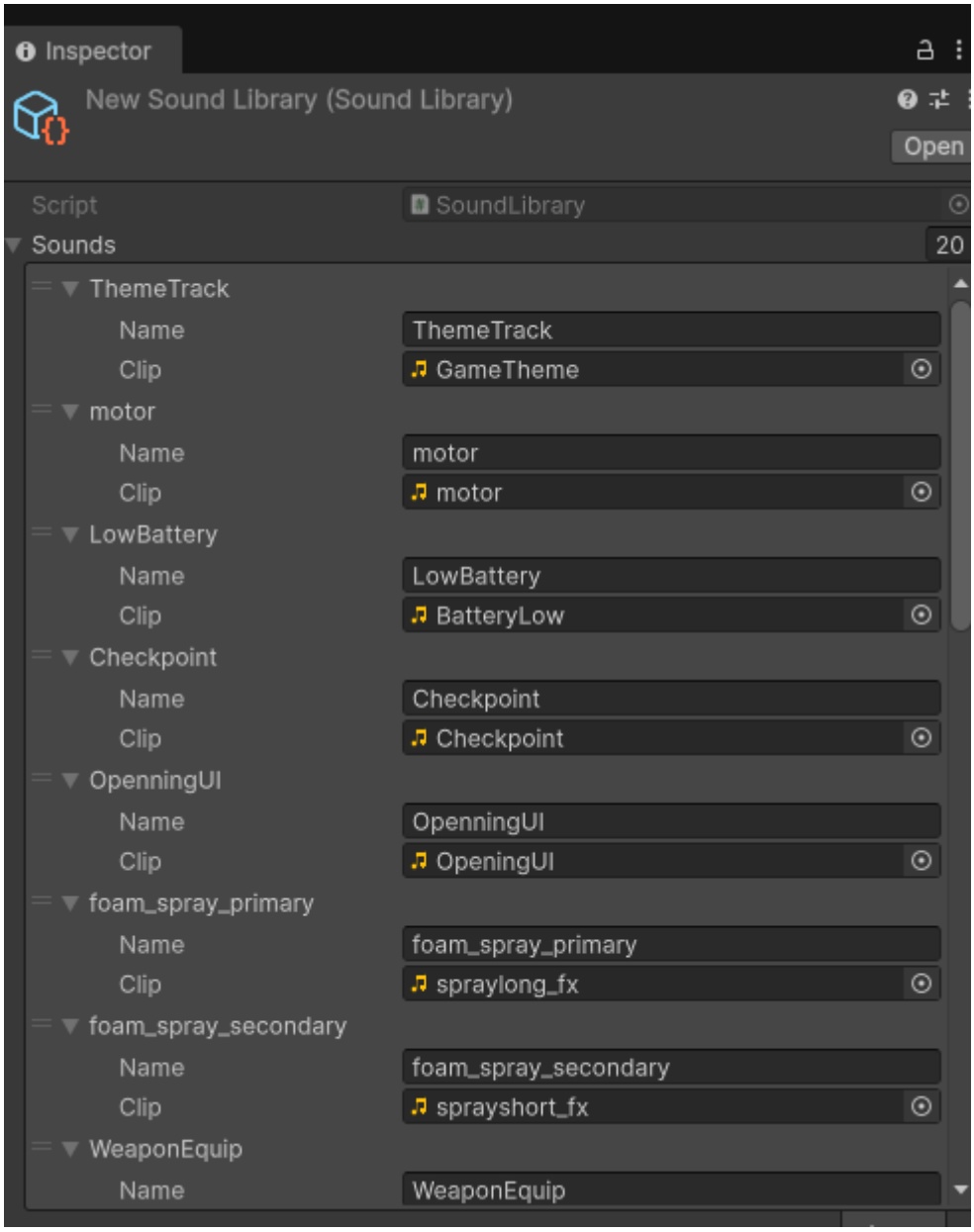
SoundLibrary is the scriptable object which will store the sound string names and audio clip.

```
[CreateAssetMenu(menuName = "Audio/Sound Library")]
public class SoundLibrary : ScriptableObject
{
    [System.Serializable]
    public class Sound
    {
        public string name;
        public AudioClip clip;
    }

    [SerializeField] private List<Sound> sounds = new();
    private Dictionary<string, AudioClip> _soundDict;

    private void OnEnable()
    {
        _soundDict = new();
        foreach (var sound in sounds)
        {
            if (!_soundDict.ContainsKey(sound.name))
                _soundDict.Add(sound.name, sound.clip);
        }
    }

    public AudioClip GetClip(string name)
    {
        return _soundDict.TryGetValue(name, out var clip) ? clip : null;
    }
}
```



The date driven lookup is used in SoundManager:

```
public void Play(string soundName)
{
    var clip = soundLibrary.GetClip(soundName);
    if (clip != null)
    {
        sfxSource.PlayOneShot(clip, _sfxVolume);
    }
}
```

1.12.3 Technical Design Flowchart



1.13 Battery Pick Up and Trigger Elevator

1.13.1 General Ideas

This battery puzzle system will be centered around PickupSystemController as a mediator. Input class BatteryInputHandler will raise an event, the controller decides whether to pick up, drop or insert the battery via BatteryHolder. Battery and slot objects will have many events so the collider will always knows the nearest interactable. When a battery is inserted into a specific BatterySlot, MovingPlatform will listen to event like OnBatteryInserted and move. UI prompts and weapon blocking will be handled via simple events and method calls. I tried to keep each script focused on single responsibility.

1.13.2 Game Programming Patterns

Heavy Observer / Event Pattern

There are a lot of events being used like OnPickupRequested, OnPlayerEnterRange(BatteryPickup), OnPlayerExitRange(BatteryPickup), OnPlayerEnterRange(BatterySlot), OnPlayerExitRange(BatterySlot), OnBatteryInserted(BatterySlot, BatteryPickup).

```
private void SubscribeToEvents()
{
    inputHandler.OnPickupRequested += HandlePickupInput;

    batteryHolder.OnBatteryPickedUp += OnBatteryPickedUp;
    batteryHolder.OnBatteryDropped += OnBatteryDropped;
    batteryHolder.OnBatteryInserted += OnBatteryInserted;

    weaponInputHandler.OnWeaponSelectionRequested += HandleWeaponSelectionAttempt;

    battery.OnPlayerEnterRange += OnBatteryRangeEnter;
    battery.OnPlayerExitRange += OnBatteryRangeExit;

    slot.OnPlayerEnterRange += OnSlotRangeEnter;
    slot.OnPlayerExitRange += OnSlotRangeExit;
}
```

Mediator Pattern

PickupSystemController is the mediator or can be called as a interaction hub that centralizes the decision making: input, holder, pickups, slots, UI, weapons. Other components stay simple and mostly event-driven.

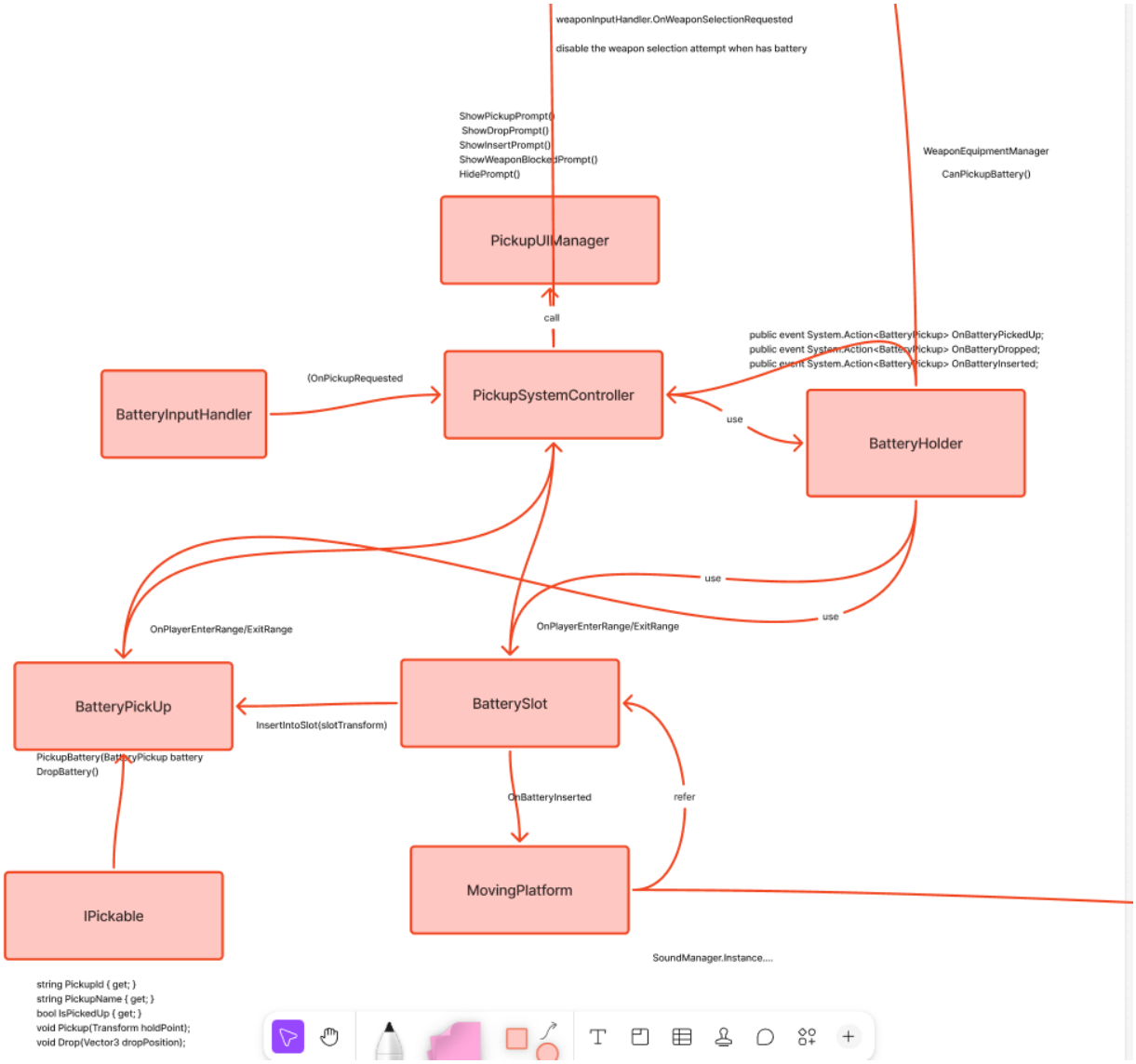
For example, PicupSystemConttroller.HandlePickupInput will make different operation based on different situation for one interaction input.

```
private void HandlePickupInput()
{
    if (batteryHolder.HasBattery && _currentNearestSlot != null && _currentNearestSlot.CanInsertBattery())
    {
        batteryHolder.InsertBattery(_currentNearestSlot);
        return;
    }

    if (batteryHolder.HasBattery)
    {
        batteryHolder.DropBattery();
        return;
    }

    if (_currentNearestBattery != null && !_currentNearestBattery.IsInsertedInSlot && batteryHolder.CanPickupBattery())
    {
        batteryHolder.PickupBattery(_currentNearestBattery);
        return;
    }
}
```

1.13.3 Technical Design Flowchart



1.14 Tutorial Instruction System

1.14.1 General Ideas

InteractionPromptManager is used as the global instance holding text, image and video and controls showing or hiding prompts with priority. ManagedInterationTrigger will attach on the trigger object in scene, detecting player enter then tell manager to show or hide UI.

1.14.2 Game Programming Pattern

Singleton Pattern

InteractionPromptManger.Instance is used for controlling the instruction system. For example, InteractionPromptManager.Instance.ShowTextPrompt(...), when the trigger need to show the text.

```
public class EnhancedInteractionPromptManager : MonoBehaviour
{
    public static EnhancedInteractionPromptManager Instance { get; private set; }

    void Awake()
    {
        if (Instance == null)
        {
            Instance = this;
            DontDestroyOnLoad(gameObject);
        }
        else
        {
            Destroy(gameObject);
            return;
        }

        InitializeUI();
    }
}
```

Example of how scripts use the singleton, global access from triggers, when player enters the trigger, it calls the manager through the instance.

```
public void ShowPrompt()
{
    if (EnhancedInteractionPromptManager.Instance == null)
    {
        Debug.LogWarning("EnhancedInteractionPromptManager.Instance is null!");
        return;
    }

    switch (promptType)
    {
        case EnhancedInteractionPromptManager.PromptType.Text:
            EnhancedInteractionPromptManager.Instance.ShowTextPrompt(promptText, priority);
            break;

        case EnhancedInteractionPromptManager.PromptType.Image:
            EnhancedInteractionPromptManager.Instance.ShowImagePrompt(promptImage, priority);
            break;

        case EnhancedInteractionPromptManager.PromptType.Video:
            EnhancedInteractionPromptManager.Instance.ShowVideoPrompt(promptVideo, priority, loopVideo);
            break;
    }
}
```

And on exit, it hides via the same singleton

```
public void HidePrompt()
{
    if (EnhancedInteractionPromptManager.Instance != null)
    {
        EnhancedInteractionPromptManager.Instance.HidePrompt(priority);
    }

    isShowingPrompt = false;
    onPromptHidden?.Invoke();
}
```

1.14.3 Technical Design Flowchart

